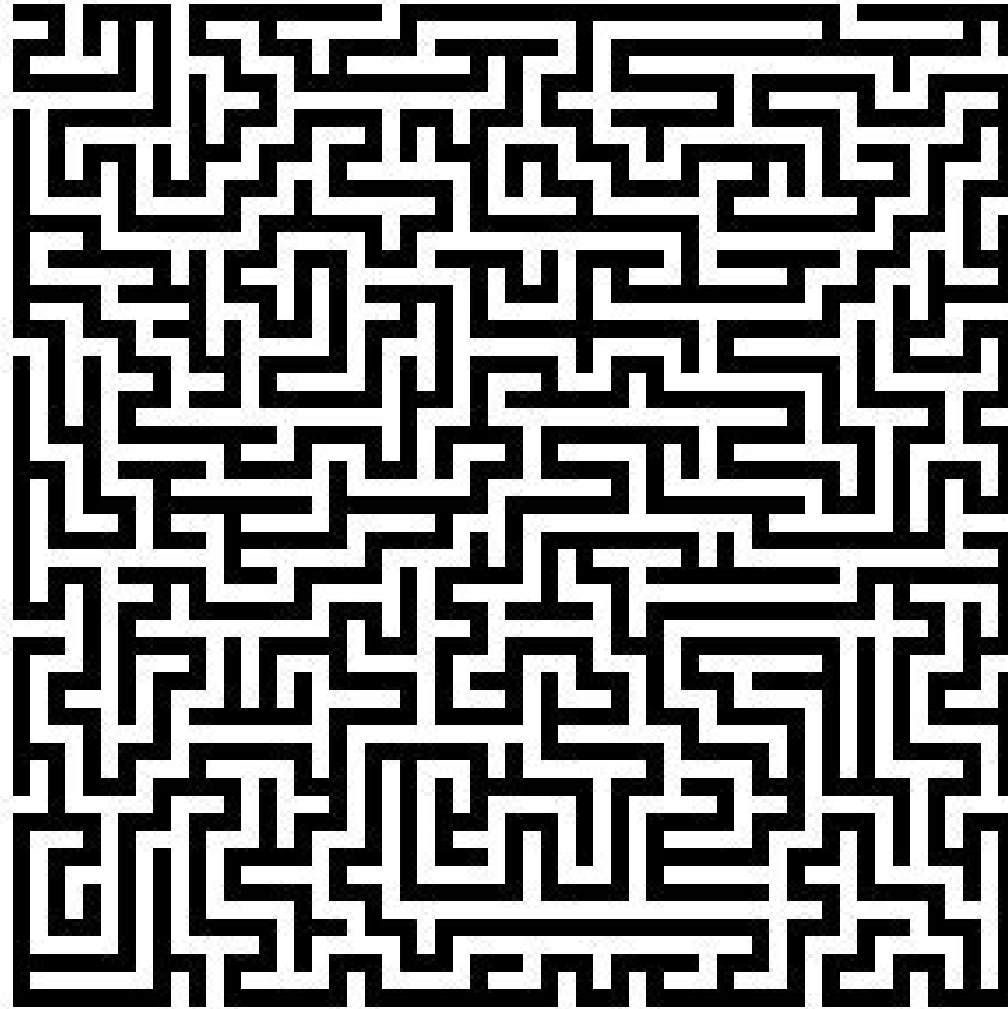


Uninformed Search

Solving problems by searching



Search as Problem-Solving Strategy

many problems: reach goal state from a given starting point

PROBLEM SOLVING FLOWCHART



source

Search as Problem-Solving Strategy

- When the correct action to take is not immediately obvious, an agent may need to **plan ahead**: to consider a **sequence of actions** that form a path to a goal state.
- This agent is called **problem-solving agent**.

Examples

getting from home to SDU

- start: home
- goal: SDU
- operators: move one block, turn

loading a moving truck

- start: apartment full of boxes and furniture
- goal: empty apartment, all boxes and furniture in the truck
- operators: select item, carry item from apartment to truck, load item

getting settled

- start: items randomly distributed over the place
- goal: satisfactory arrangement of items
- operators: select item, move item

Motivation



search strategies - important methods for many approaches to problem-solving



to use search - abstract formulation of the problem and the available steps



search algorithms - basis for many optimization and planning methods

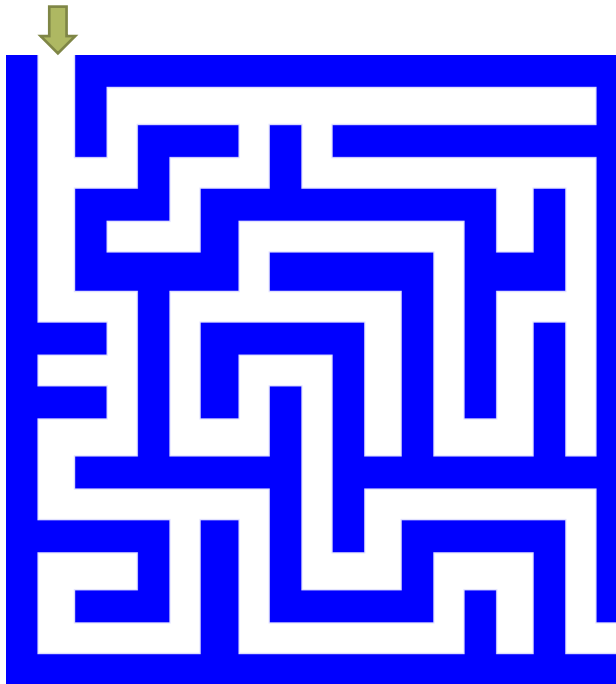
Objectives

- formulate appropriate problems as search tasks
 - states, initial state, goal state, successor functions (operators), cost
- know the fundamental search strategies and algorithms
 - uninformed search
 - breadth-first, depth-first, uniform-cost, iterative deepening
 - informed search
 - best-first (greedy, A^*), heuristics
- evaluate the suitability of a search strategy for a problem
 - completeness, optimality

Search

- We will consider the problem of designing **goal-based agents** in **observable, deterministic, discrete, known** environments
- Example:

Start state



Goal state

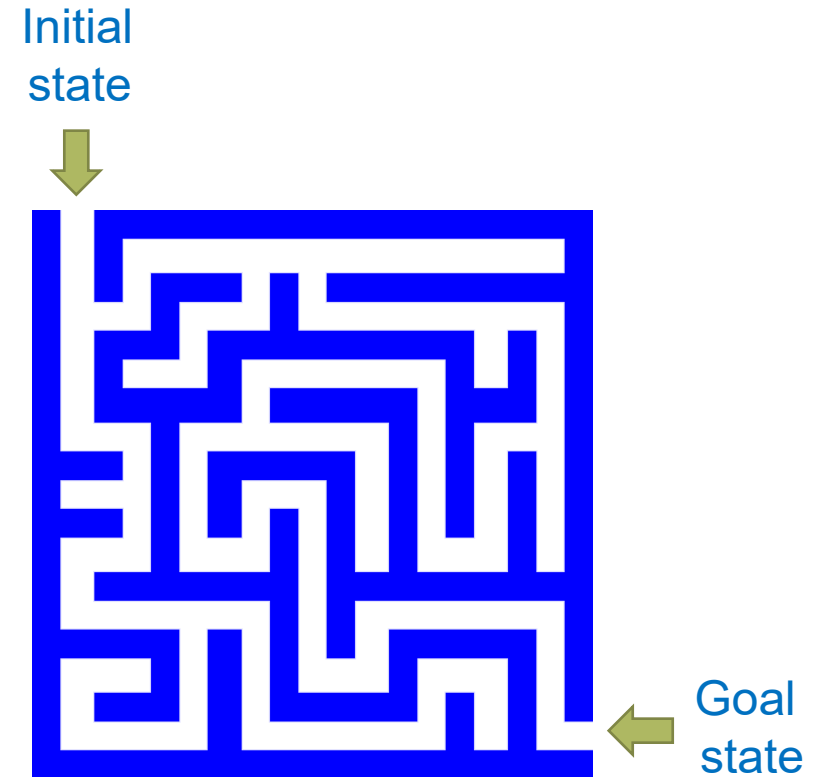
Search

We will consider the problem of designing **goal-based agents** in **observable, deterministic, discrete, known** environments

- **Solution**: fixed sequence of actions
- **Search**: process of looking for the sequence of actions that reaches the goal
- **Agent** can ignore percepts during execution (can close his/her eyes!)

Search problem components

- **Initial state**
- **Successor Function (Operator)**
 - Result of doing an action in a state
- **Goal state**
- **Path cost**
 - Assume that it is a sum of nonnegative *step costs*
- **Optimal solution:** sequence of actions with the **lowest path cost** for reaching the goal



Example: Romania

- On vacation in Romania; currently in Arad
- Flight leaves tomorrow from Bucharest

- **Initial state**

- E.g. Arad

- **Successor Function**

- $S(\text{Arad}) = \{\text{Zerind}, \text{Timisoara}, \text{Sibiu}\}$

- **Goal state**

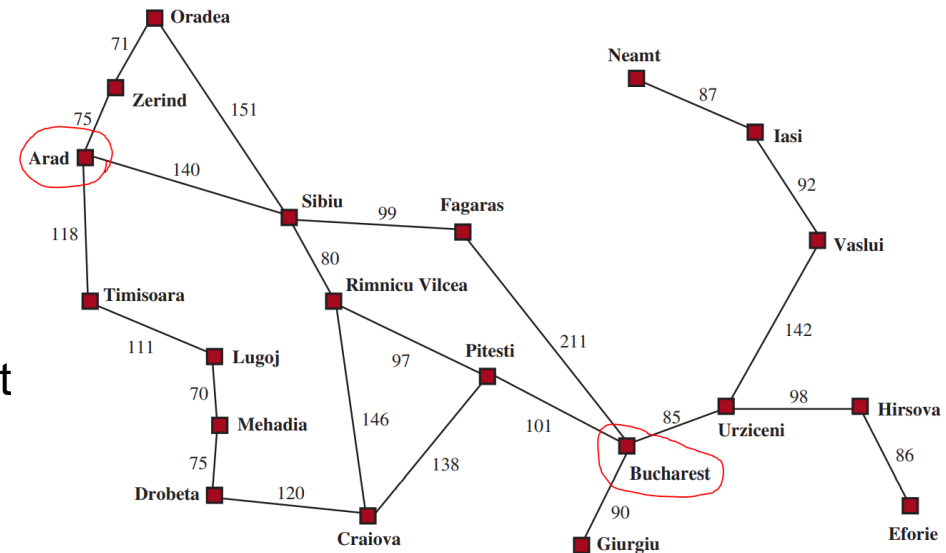
- Bucharest

- **Path Cost**

- Sum of edge costs

- **A solution is**

- a sequence of actions leading from the initial state to a goal state



State Space

- The initial state and the successor function define the **state space** of the problem
 - The set of all states reachable from initial state by any sequence of actions
 - Can be represented as a **directed graph** (nodes are states and links between nodes are actions)
- What is the state space for the Romania problem?

Example: Vacuum world

Initial State

- Any

Goal State

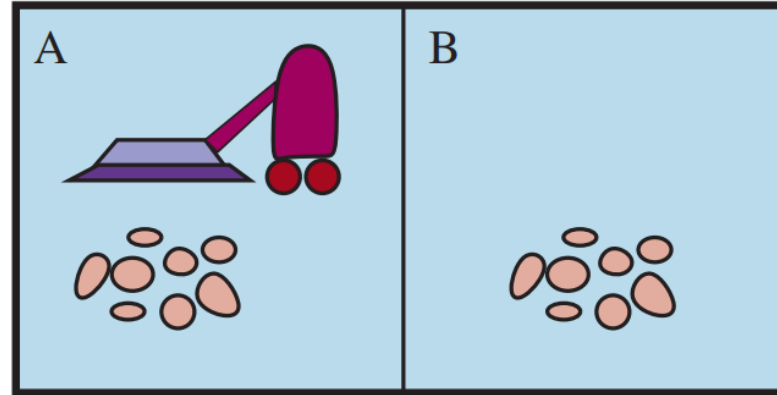
- All clean

Successor function

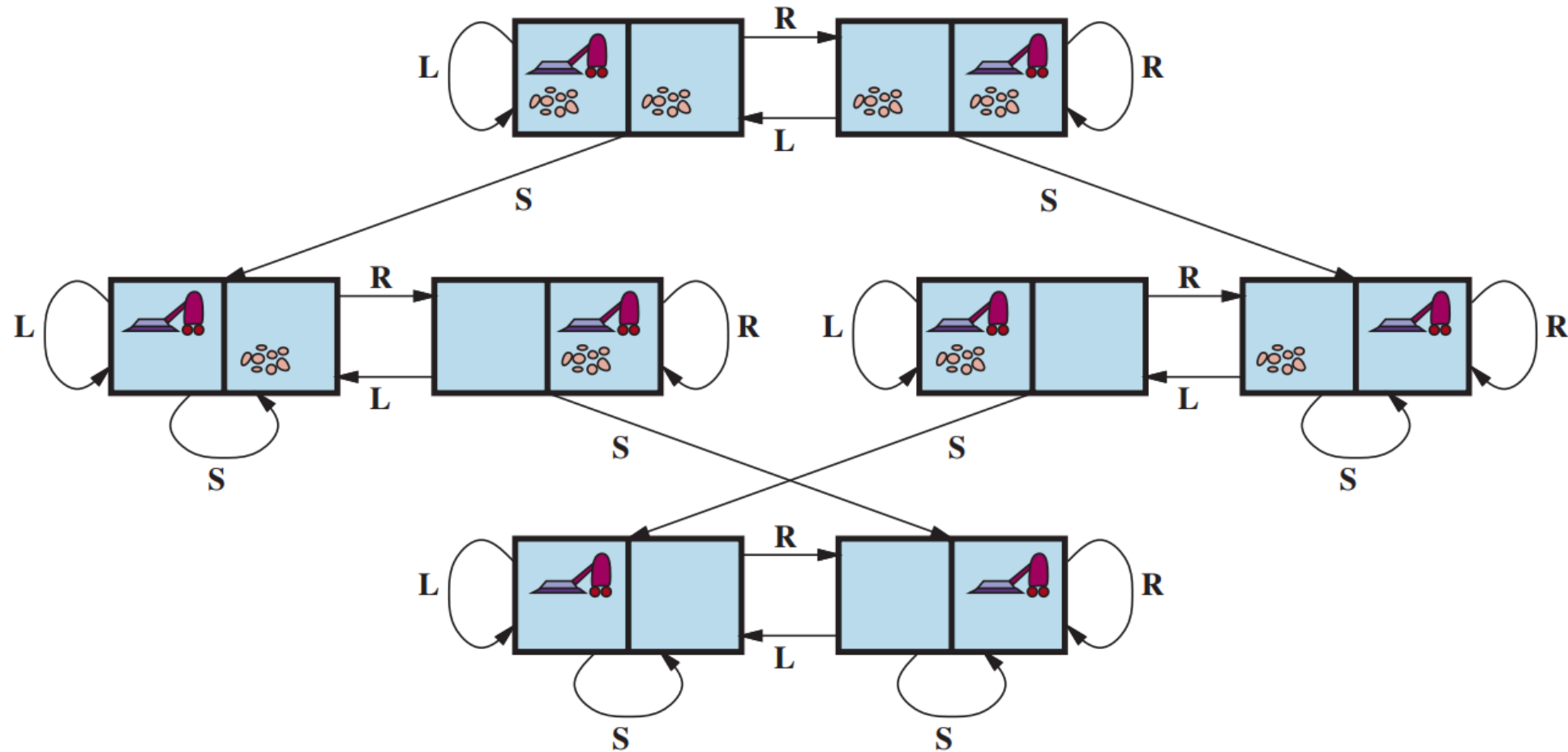
- Described by state space (next slide)

Path Cost

- Could be the sum of the amounts of electricity consumed with each move



Vacuum world state space graph



Example: The 8-puzzle

Initial State

- Any locations of tiles, number of states:
 - 8-puzzle: $9!/2 = 181,440$ states
 - 15-puzzle: $16!/2 > 10$ trillion states
 - 24-puzzle: 10^{25} states

Successor Function

- Actions:** Move blank left, right, up, down, and consequent states

Path cost

- 1 per move

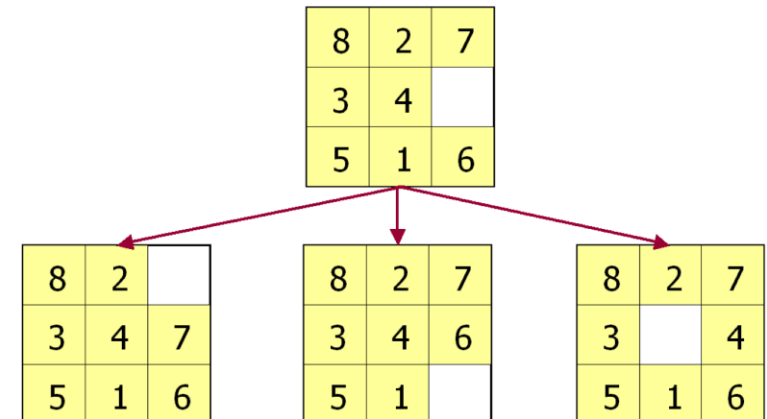
Optimal solution of n-Puzzle is **NP-hard**

7	2	4
5		6
8	3	1

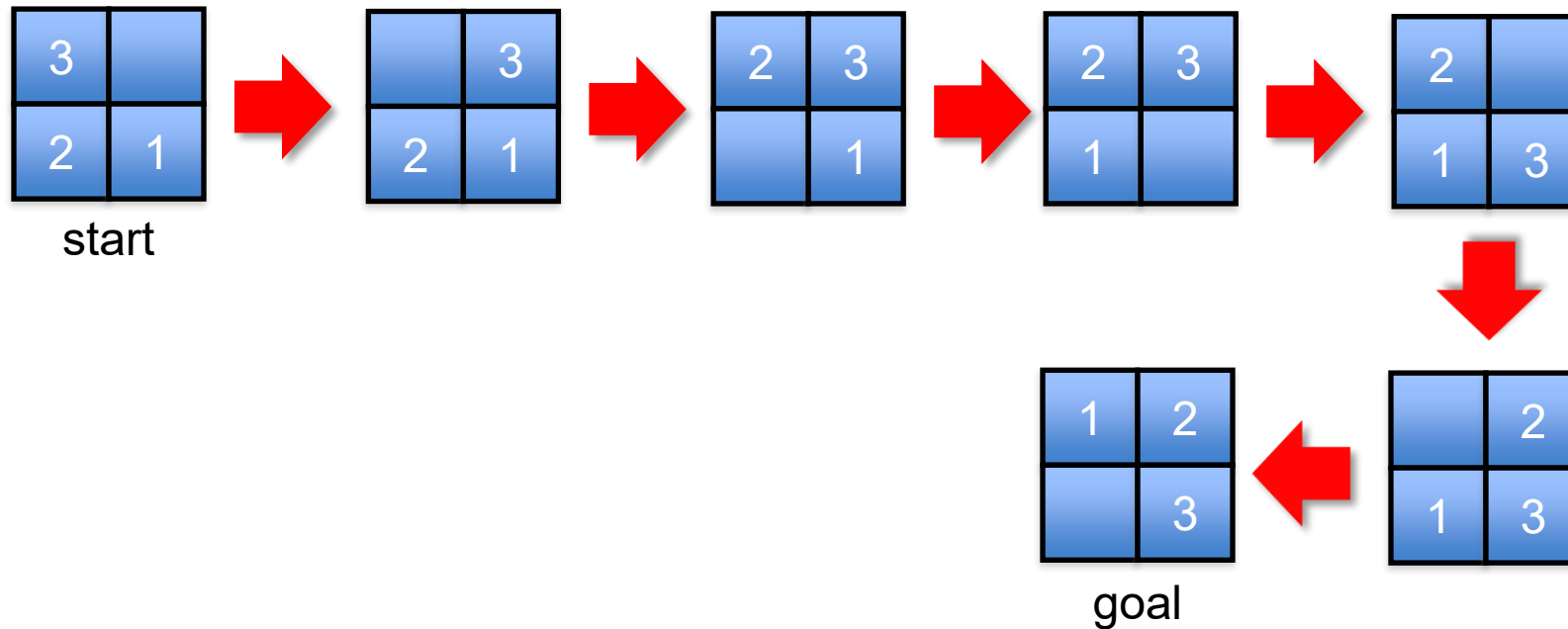
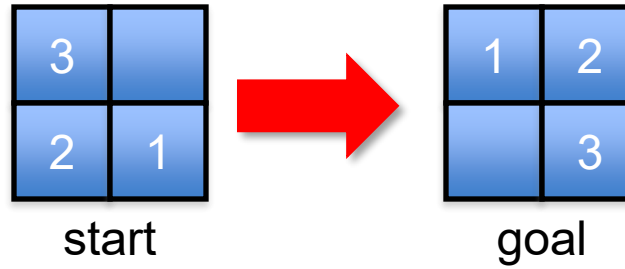
Start State

	1	2
3	4	5
6	7	8

Goal State



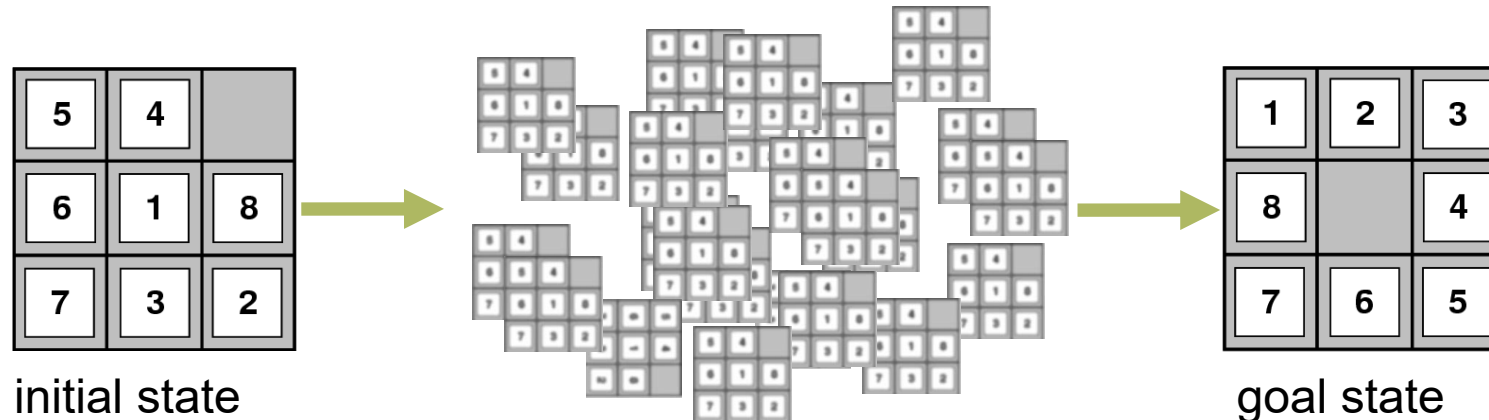
Simpler: 3-Puzzle



Building goal-based agents

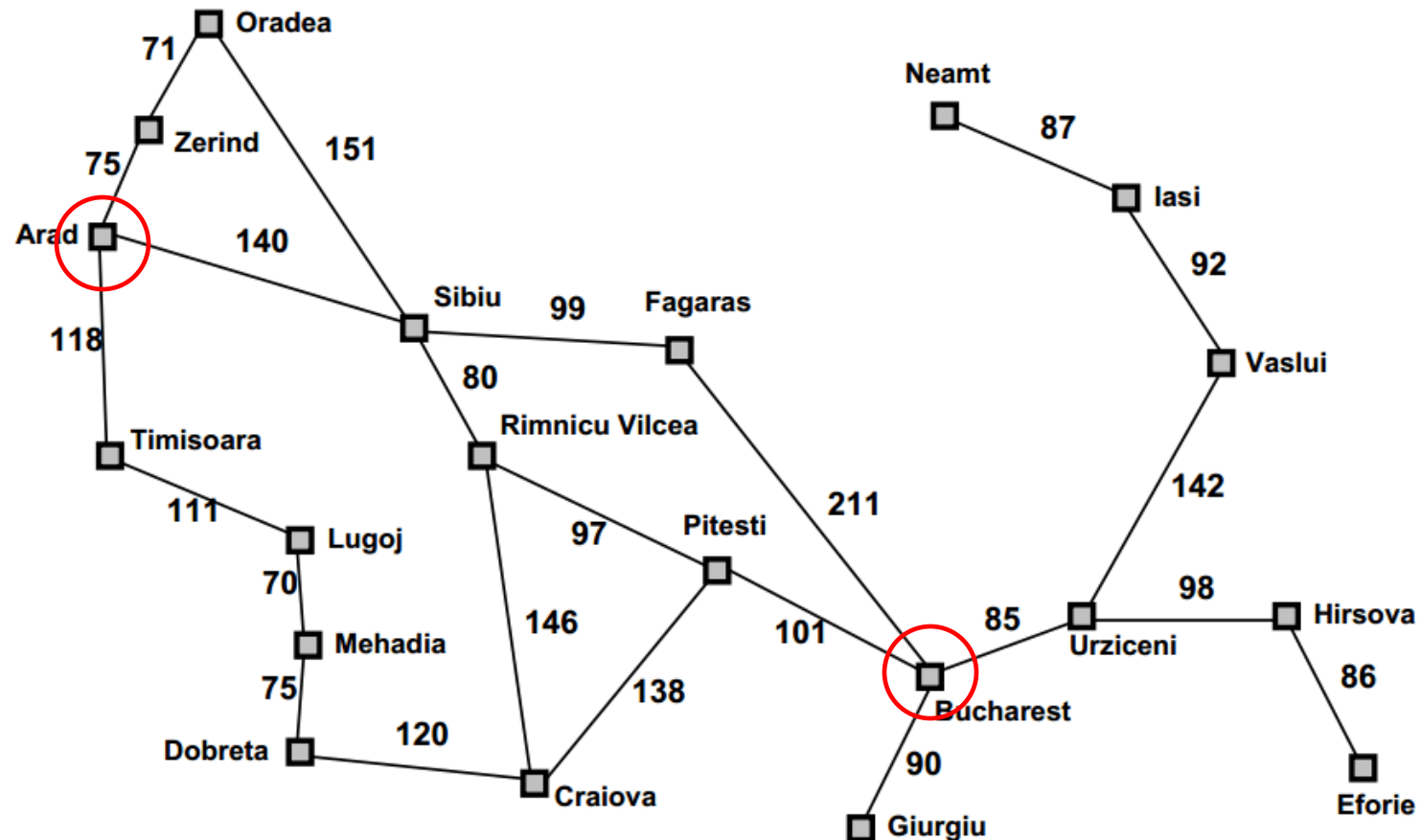
We must answer the following questions

- How do we represent the **state** of the “world”?
- What is the **goal** and how can we recognize it?
- What are the possible **actions**?
- What *relevant* information do we encode to describe states, actions and their effects and thereby solve the problem?



Example: Route Planning

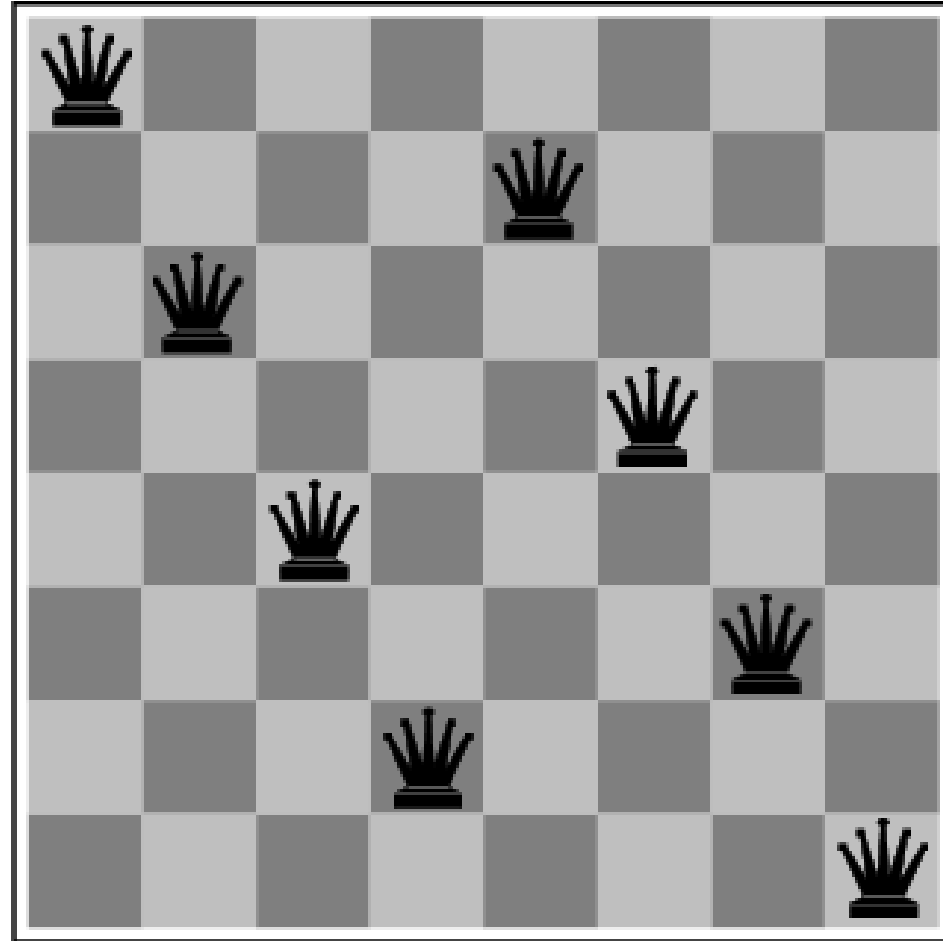
Find a route from Arad to Bucharest



Example: The 8-Queens Puzzle

Place eight queens on a chessboard such that no queen attacks any other

We can generalize the problem to a $N \times N$ chessboard



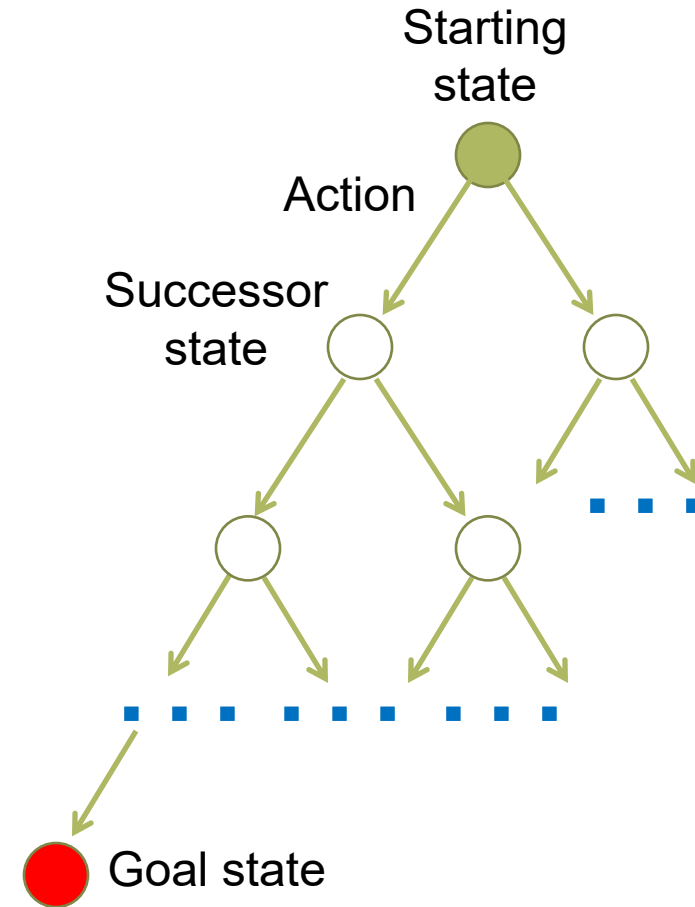
What are the states, successor function, goal state?

Tree Search

- Begin at the root node (starting state) and **expand** it by making a list of all possible successor states
- Maintain a **fringe** or a list of unexpanded states
- At each step, pick a state from the fringe to expand
- Keep going until you reach the goal state
- Try to expand as few states as possible
- A solution is a path ending in the goal state

Nodes vs. states

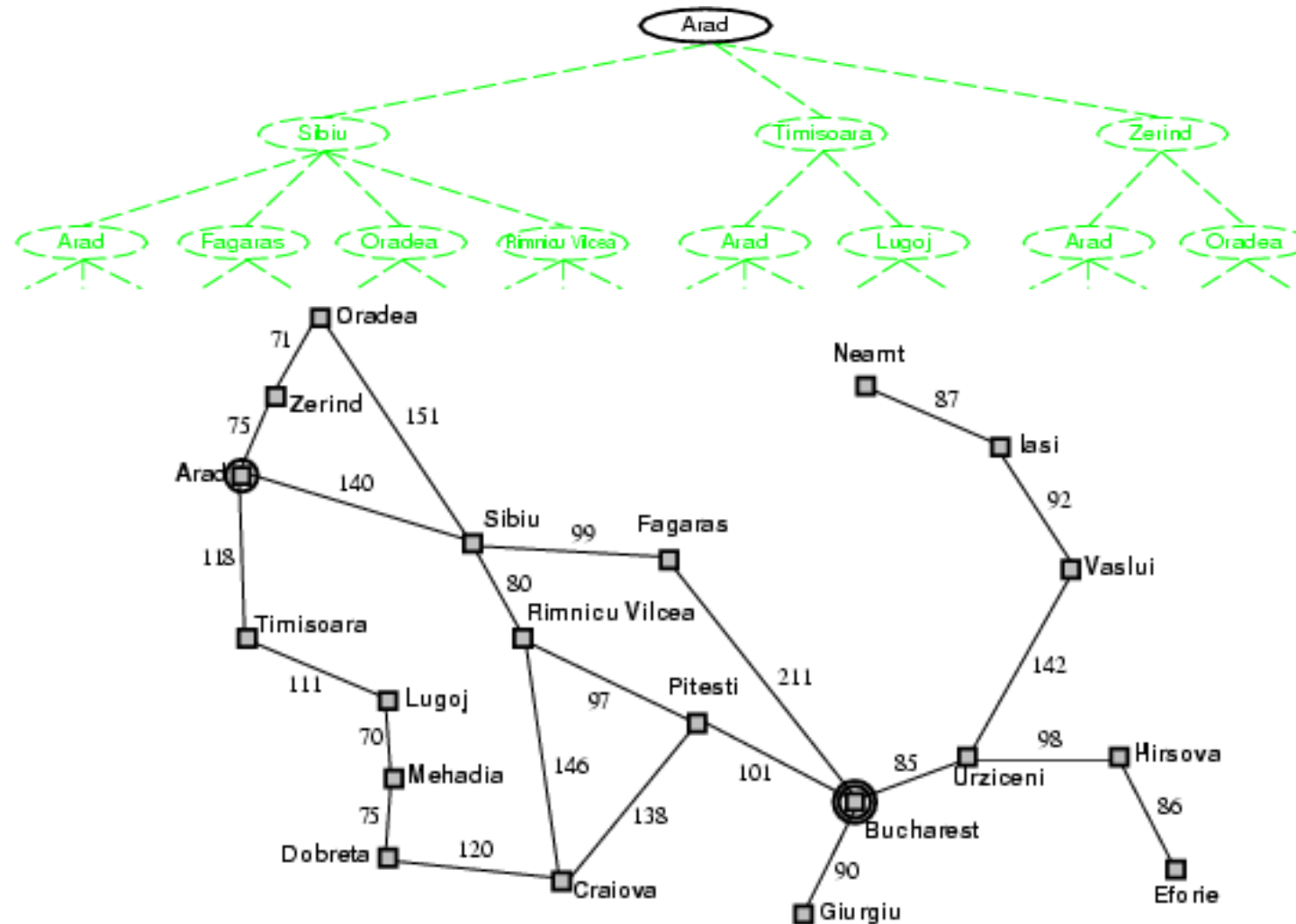
- A state is a representation of a physical configuration, while a node is a data structure that is part of the search tree



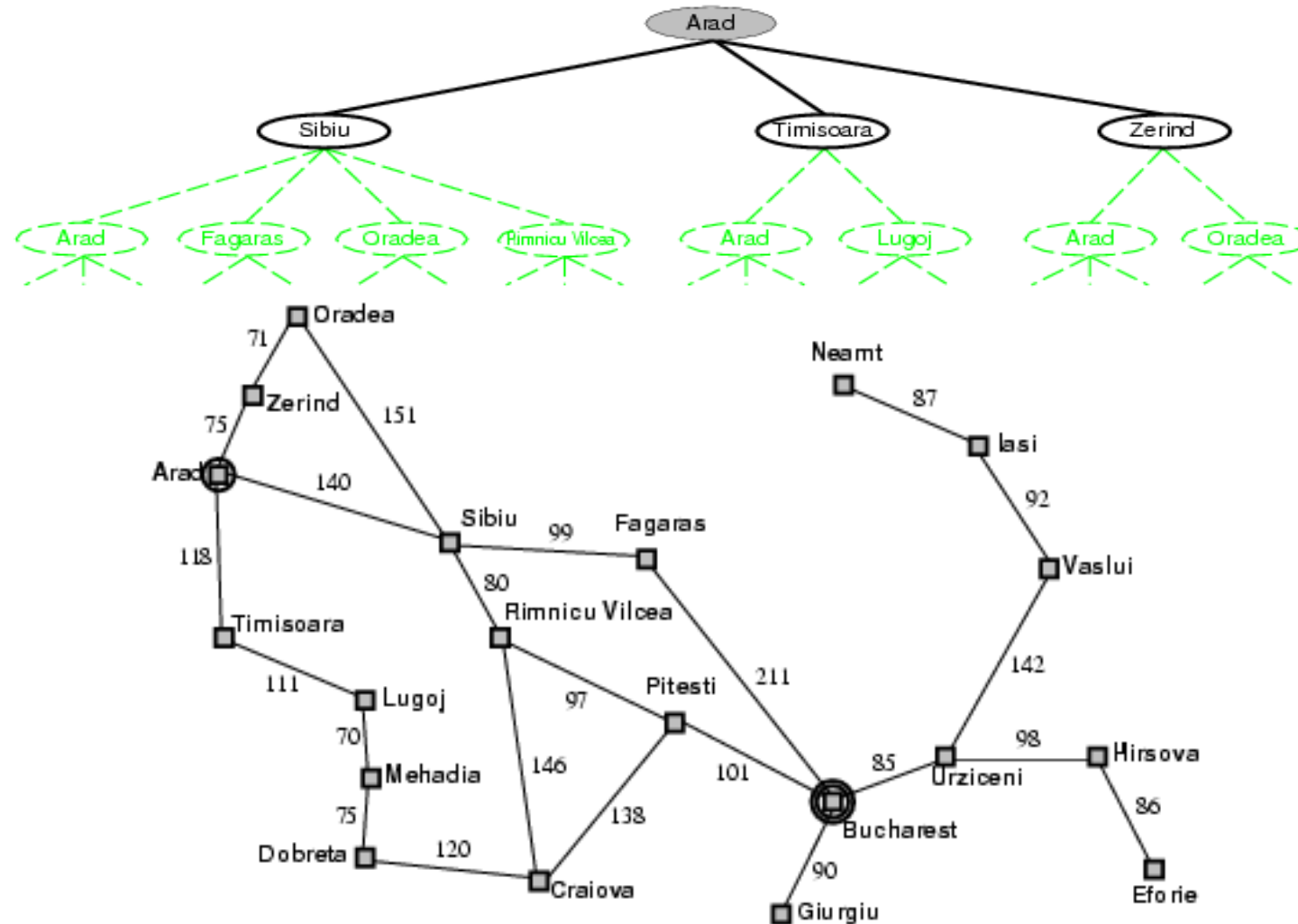
Tree Search Algorithm Outline

- Initialize the **fringe** using the **starting state**
- While the fringe is not empty
 - Choose a fringe node to expand according to **search strategy**
 - If the node contains the **goal state**, return solution
 - Else **expand** the node and add its children to the fringe
- To handle repeated states:
 - Keep an **explored set**; add each node to the explored set every time you expand it
 - Every time you add a node to the fringe, check whether it already exists in the fringe with a higher path cost, and if yes, replace that node with the new one

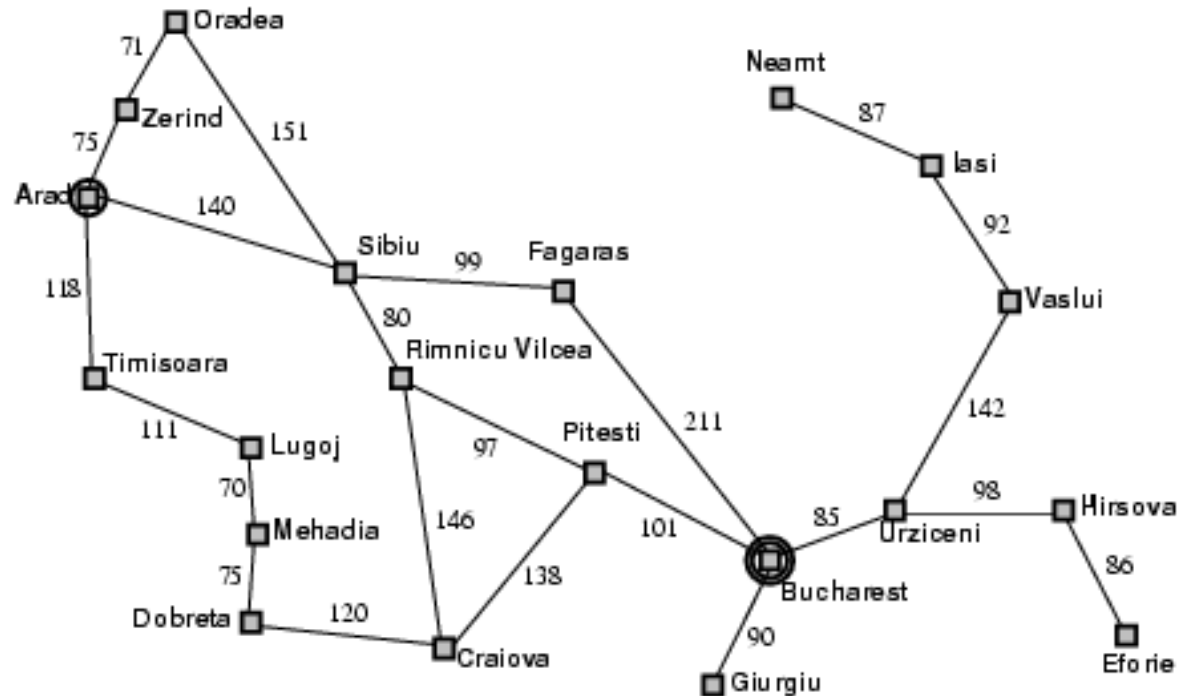
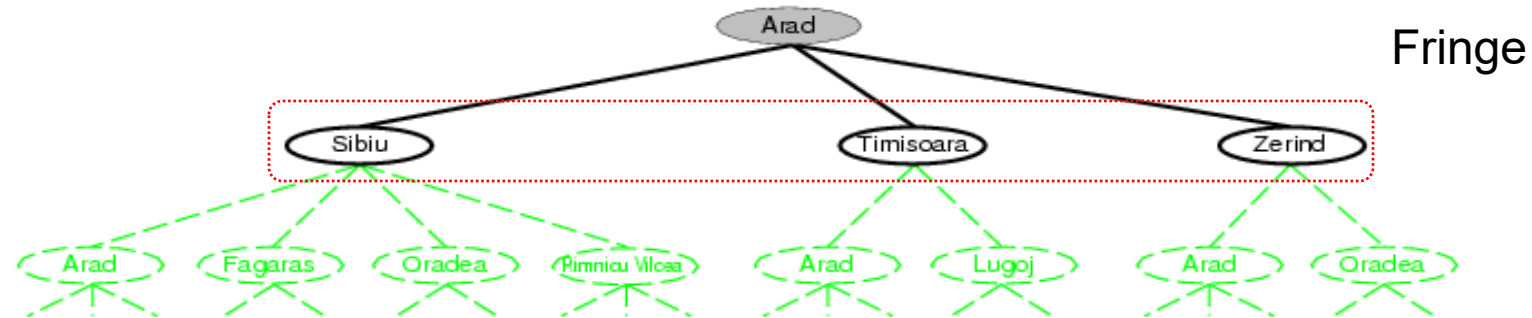
Tree search example



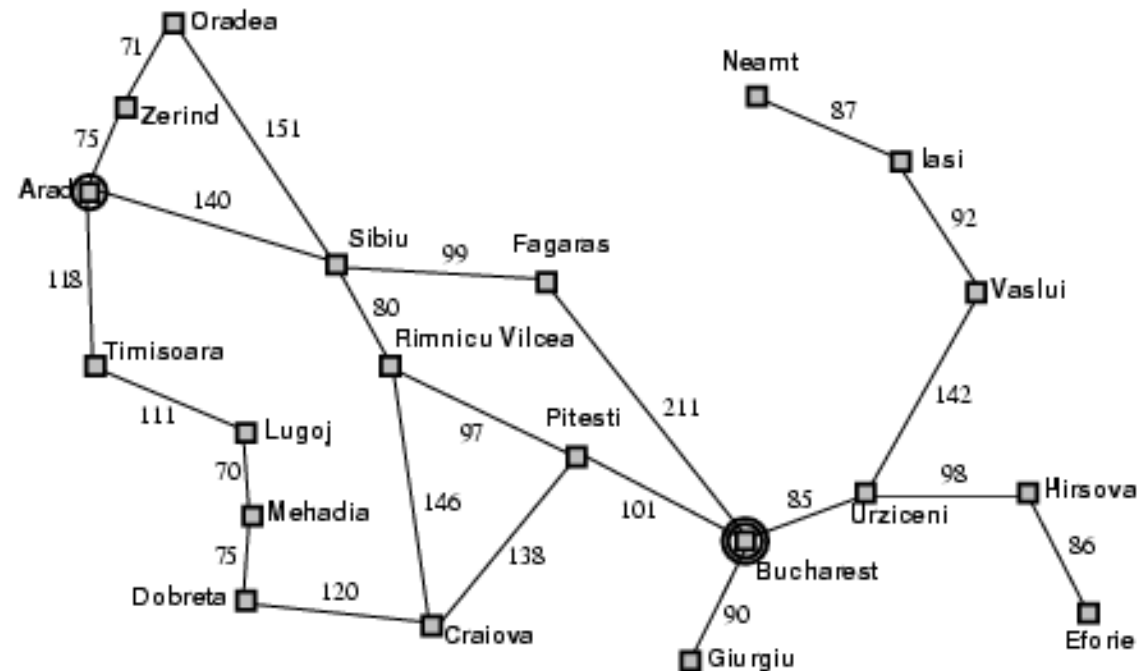
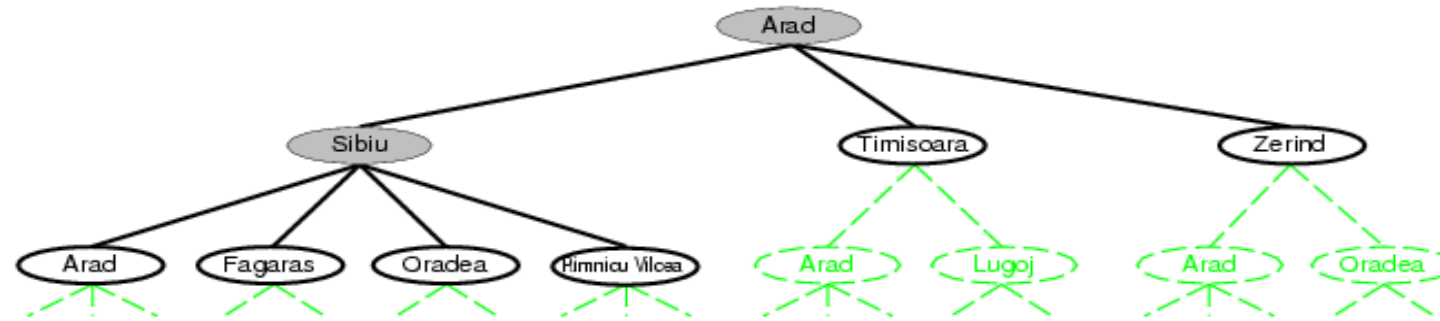
Tree search example



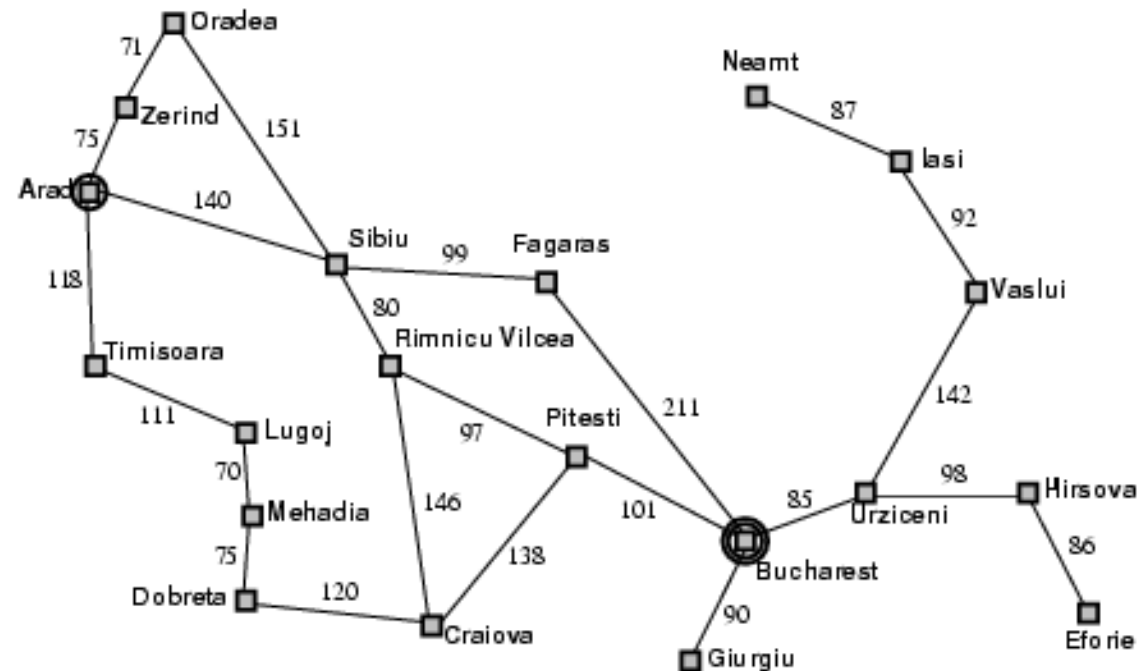
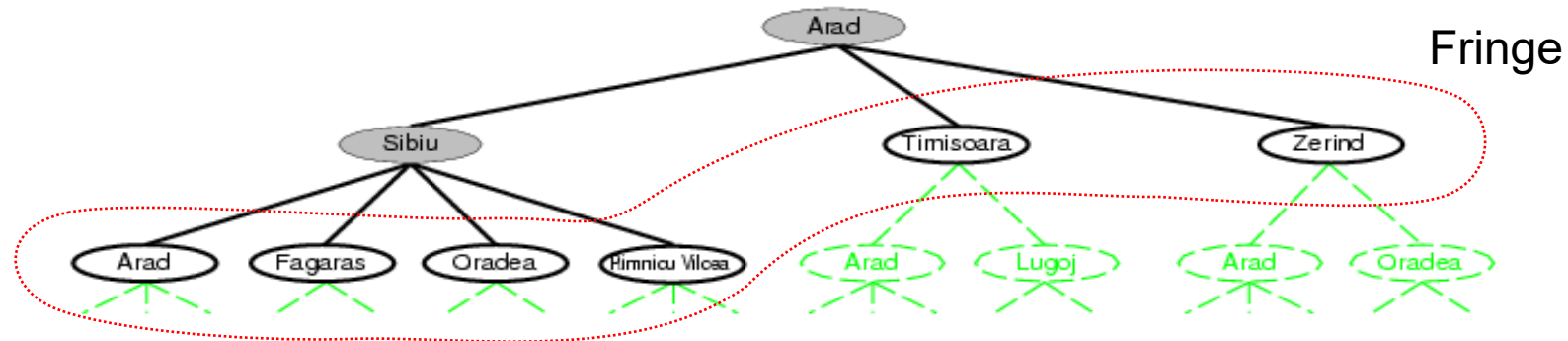
Tree search example



Tree search example



Tree search example



Search strategies

A **search strategy** is defined by picking the order of node expansion

Strategies are evaluated along the following dimensions:

- **Completeness:** does it always find a solution if one exists? and to correctly report failure when there is not?
- **Optimality:** does it always find a least-cost solution?
- **Time complexity:** How long does it take to find the solution? (Can be measured in seconds or the number of nodes generated)
- **Space complexity:** How much memory is needed to perform the search (maximum number of nodes in memory)

Search strategies

Time and space complexity are measured in terms of

- b : maximum branching factor of the search tree
- d : depth of the least-cost (cheapest) solution
- m : maximum length of any path in the state space (may be infinite)

Uninformed search strategies

Uninformed search strategies use only the information available in the problem definition

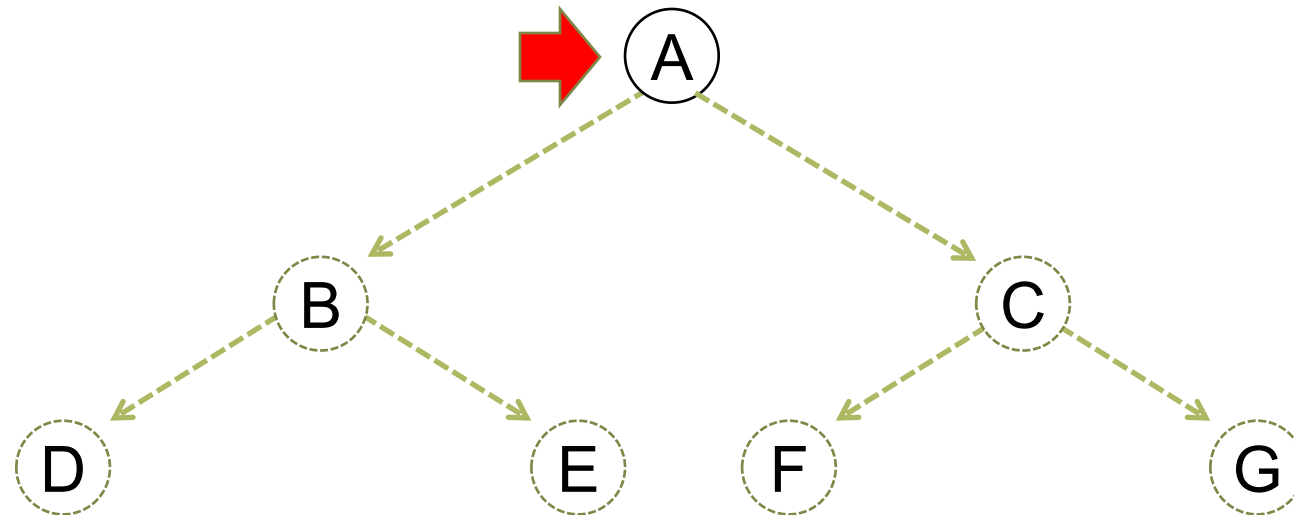
- Breadth-first search (BFS)
- Uniform-cost search (UCS)
- Depth-first search (DFS)
- Iterative deepening search (IDS)

Breadth-first search

Expand shallowest unexpanded node

Implementation:

- *fringe* is a FIFO queue, i.e., new successors go at end

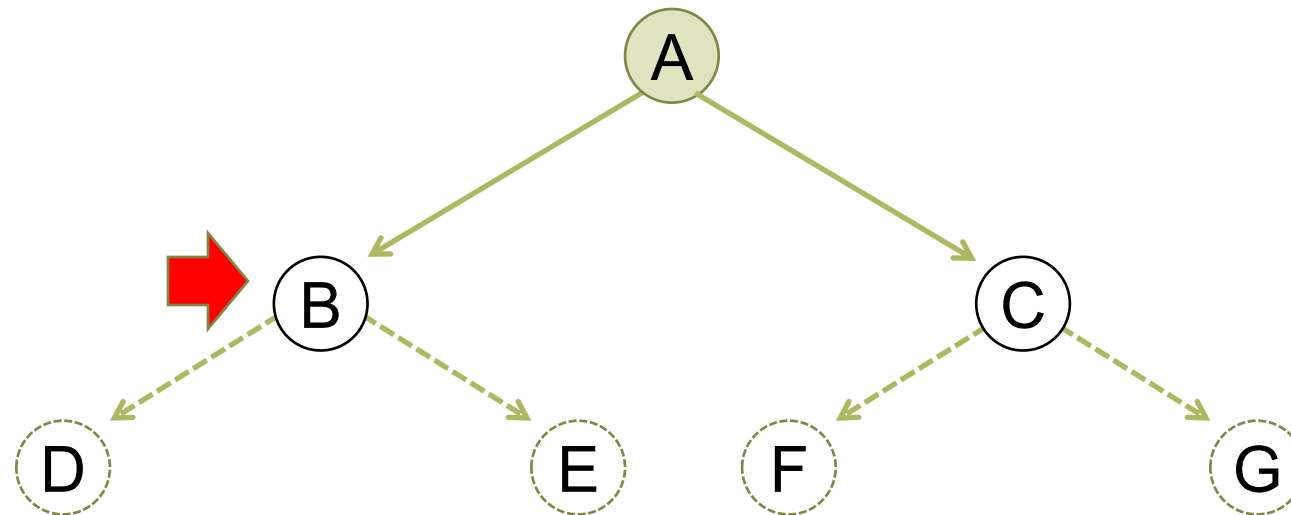


Breadth-first search

Expand shallowest unexpanded node

Implementation:

- *fringe* is a FIFO queue, i.e., new successors go at end

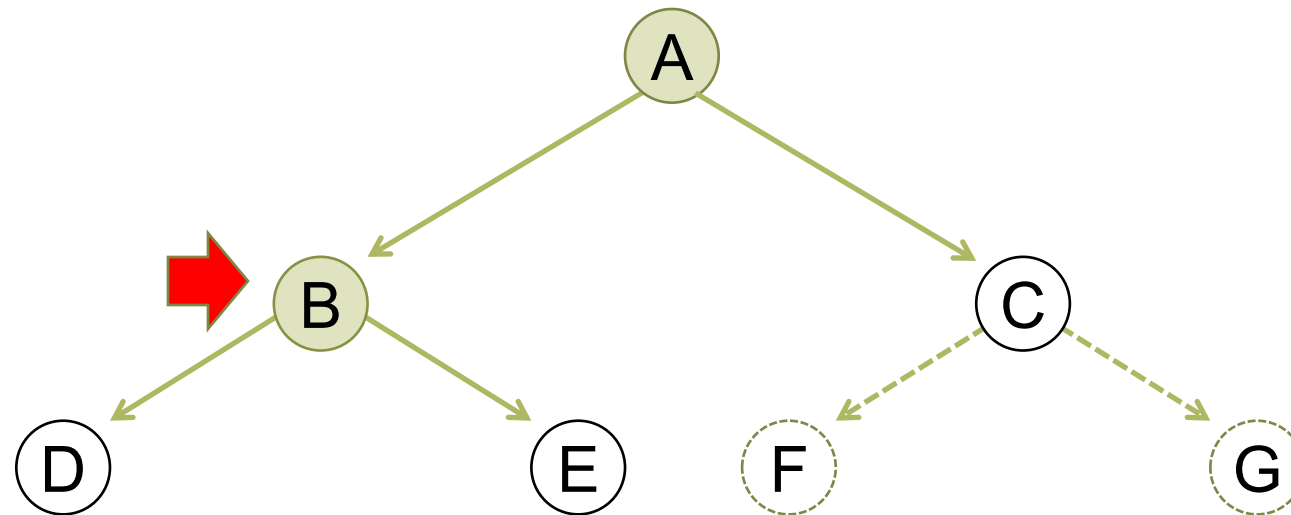


Breadth-first search

Expand shallowest unexpanded node

Implementation:

- *fringe* is a FIFO queue, i.e., new successors go at end

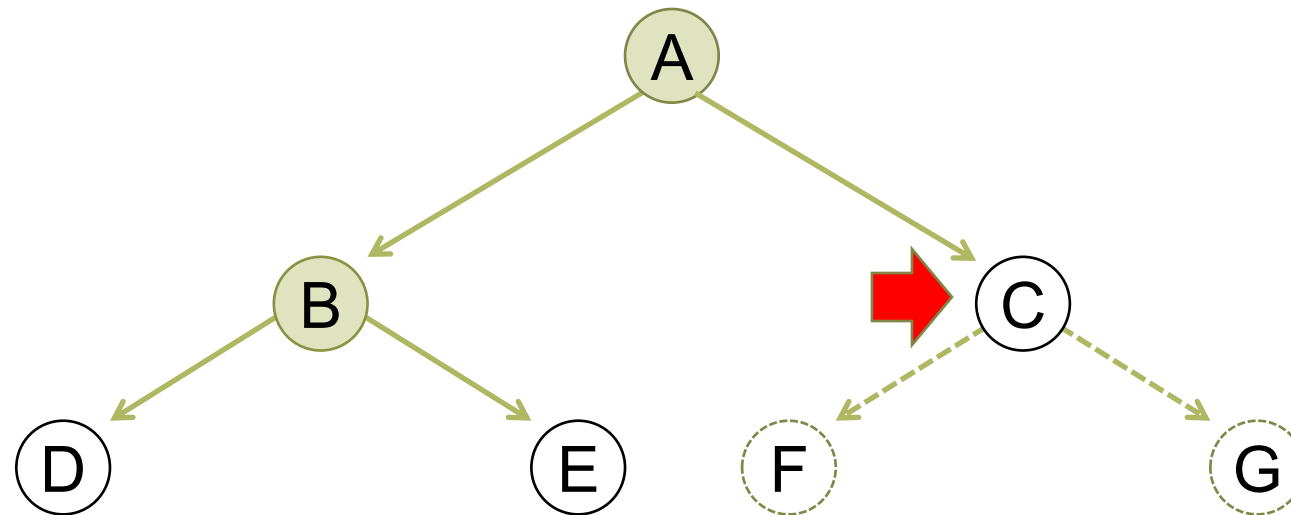


Breadth-first search

Expand shallowest unexpanded node

Implementation:

- *fringe* is a FIFO queue, i.e., new successors go at end

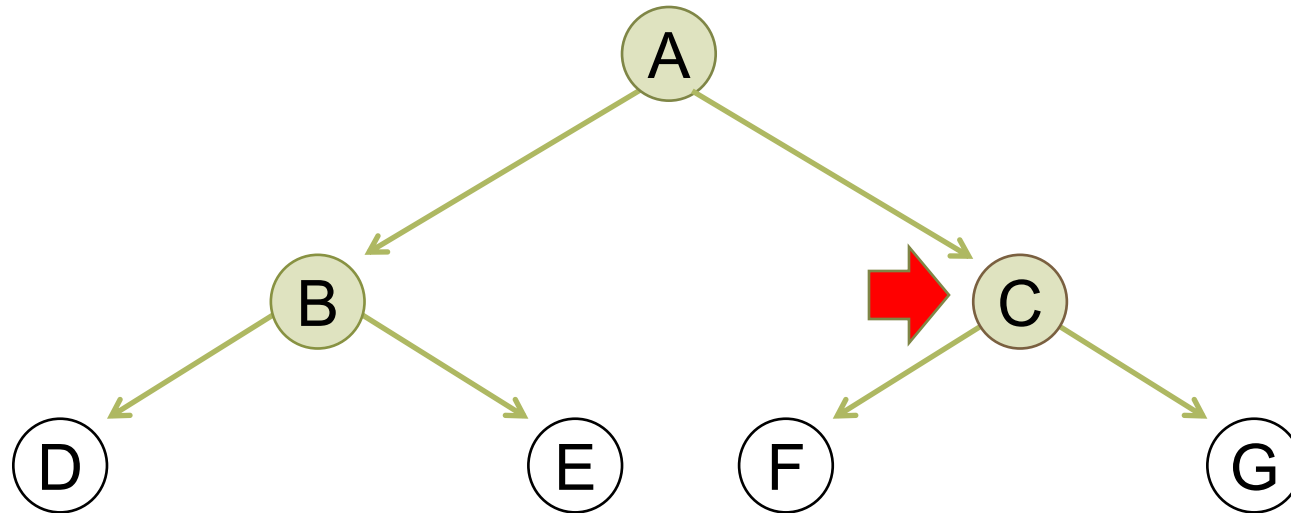


Breadth-first search

Expand shallowest unexpanded node

Implementation:

- *fringe* is a FIFO queue, i.e., new successors go at end



Properties of breadth-first search

- **Complete?**

- Yes

- **Optimal?**

- Yes – if cost = 1 per step

- **Time?**

- Number of nodes in a b -ary tree of depth d : $O(b^d)$
(d is the depth of the optimal solution)

- **Space?**

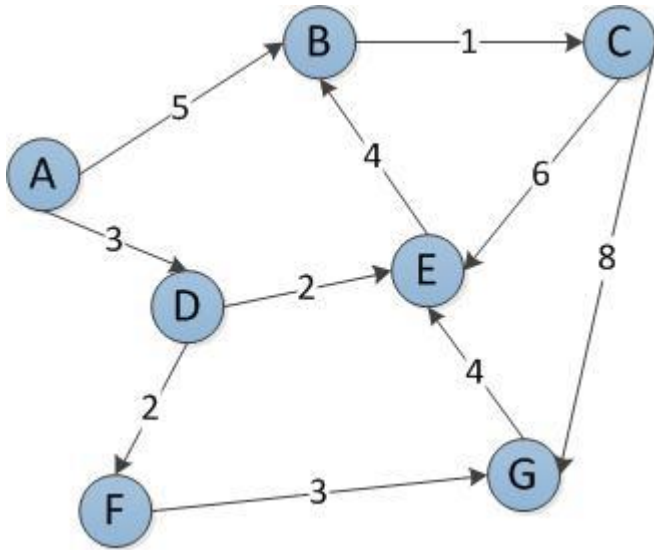
- $O(b^d)$

- Space is the bigger problem (more than time)
- In general, exponential-complexity search problems cannot be solved by uninformed search for any but the smallest instances.

Uniform-cost search

- Expand least-cost unexpanded node
- Implementation: *fringe* ordered by path cost (priority queue)
- Equivalent to breadth-first if step costs all equal
- **Complete?**
 - Yes
- **Optimal?**
 - Yes – nodes expanded in increasing order of path cost
- **Time?**
 - $O(b^{1+\lceil C^*/\epsilon \rceil})$
 - Number of nodes with path cost $\leq$ cost of optimal solution
 - C^* : cost of optimal solution, and every step cost is at least $\epsilon > 0$
 - This can be greater than b^d : the search can explore long paths consisting of small steps before exploring shorter paths consisting of larger steps
- **Space?**
 - $O(b^{1+\lceil C^*/\epsilon \rceil})$

Uniform-Cost Search Example



Start Node: A

Goal Node: G

Step 1

Fringe:

Node A

Cost 0

Explored: -

Step 2

Expand A

Fringe:

Node D B

Cost 3 5

Explored: A

Step 3

Expand D

Fringe:

Node B E F

Cost 5 5 5

Explored: A D

Step 4

Expand B

Fringe:

Node E F C

Cost 5 5 6

Explored: A D B

Step 5

Expand E

Fringe:

Node F C

Cost 5 6

Explored: A D B E

Step 6

Expand F

Fringe:

Node C G

Cost 6 8

Explored: A D B E F

Step 7

Expand C

Fringe:

Node G

Cost 14

Explored: A D B E F C

Step 8

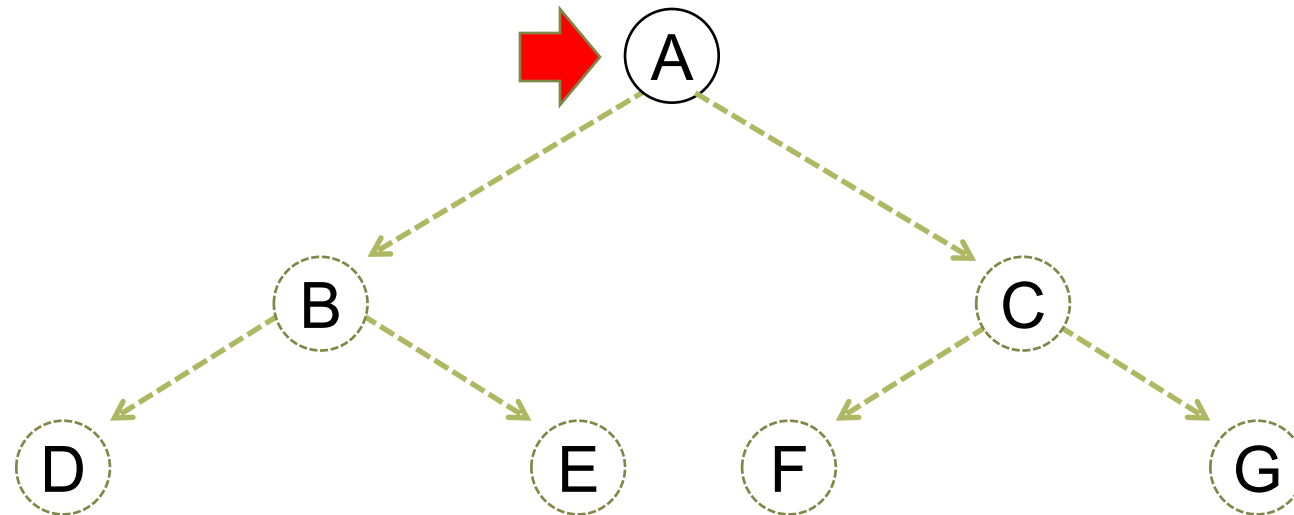
Expand G

Found the path:

A to D to F to G

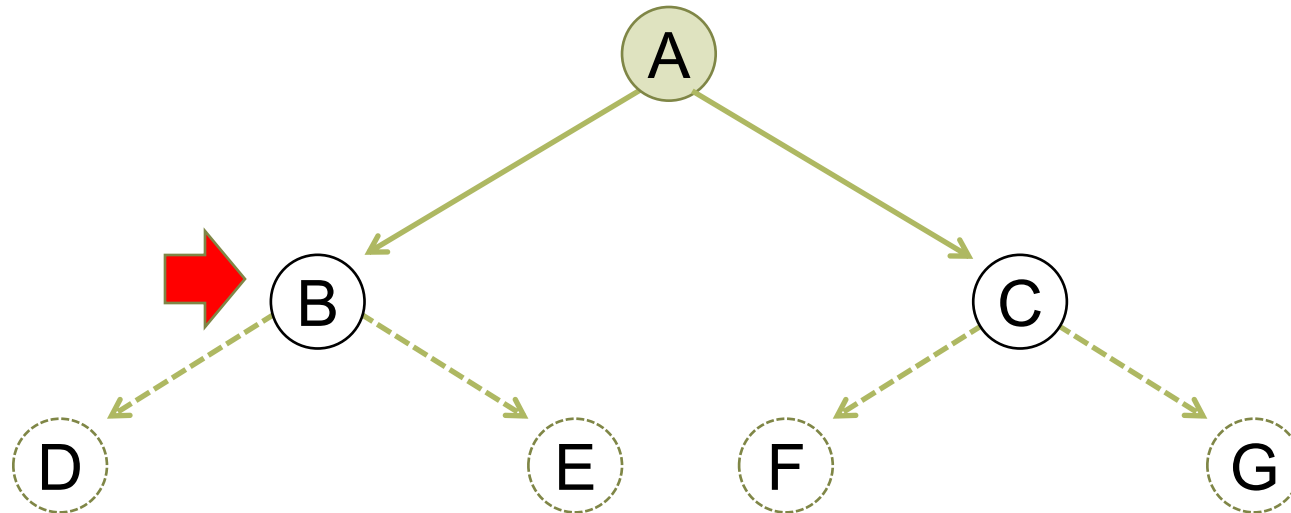
Depth-first search

- Expand deepest unexpanded node
- Implementation:
 - *fringe* = LIFO queue, i.e., put successors at front



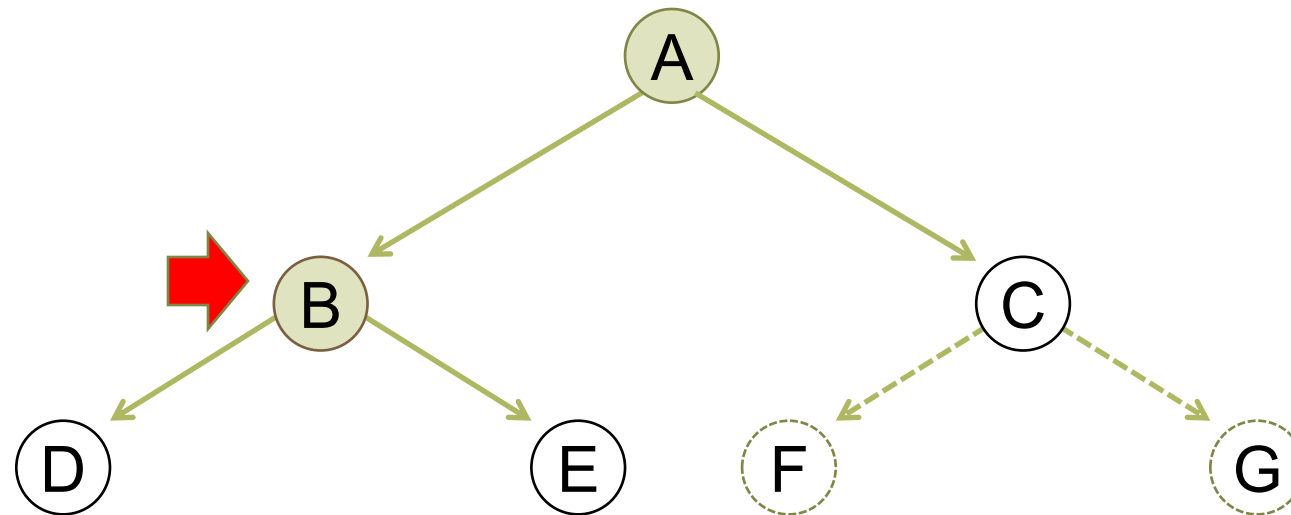
Depth-first search

- Expand deepest unexpanded node
- Implementation:
 - *fringe* = LIFO queue, i.e., put successors at front



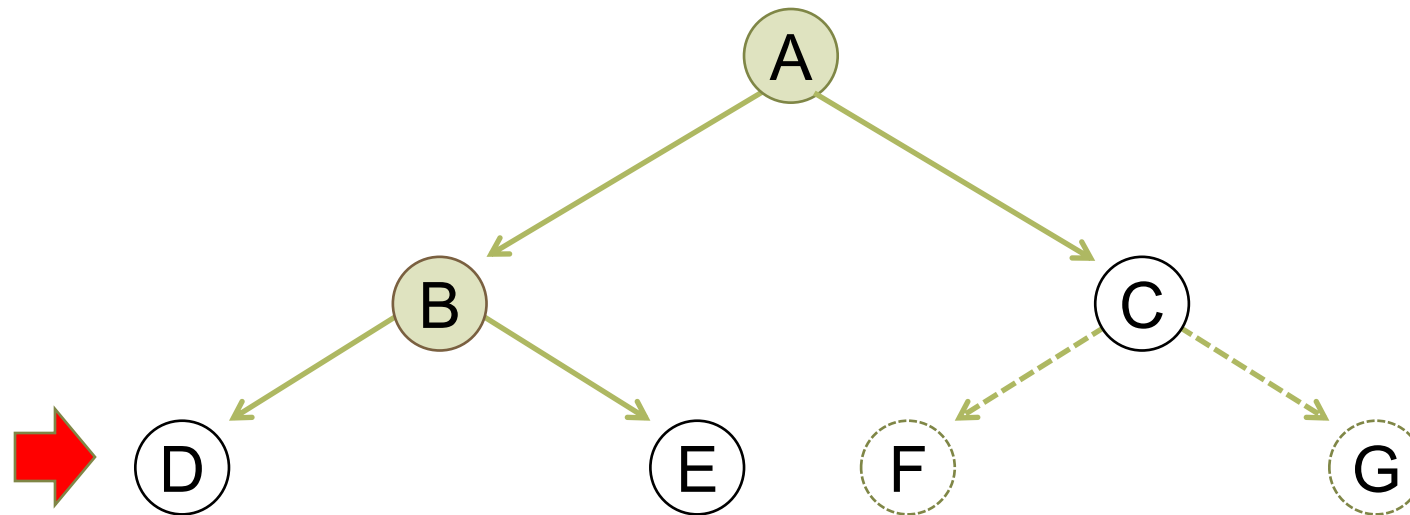
Depth-first search

- Expand deepest unexpanded node
- Implementation:
 - *fringe* = LIFO queue, i.e., put successors at front



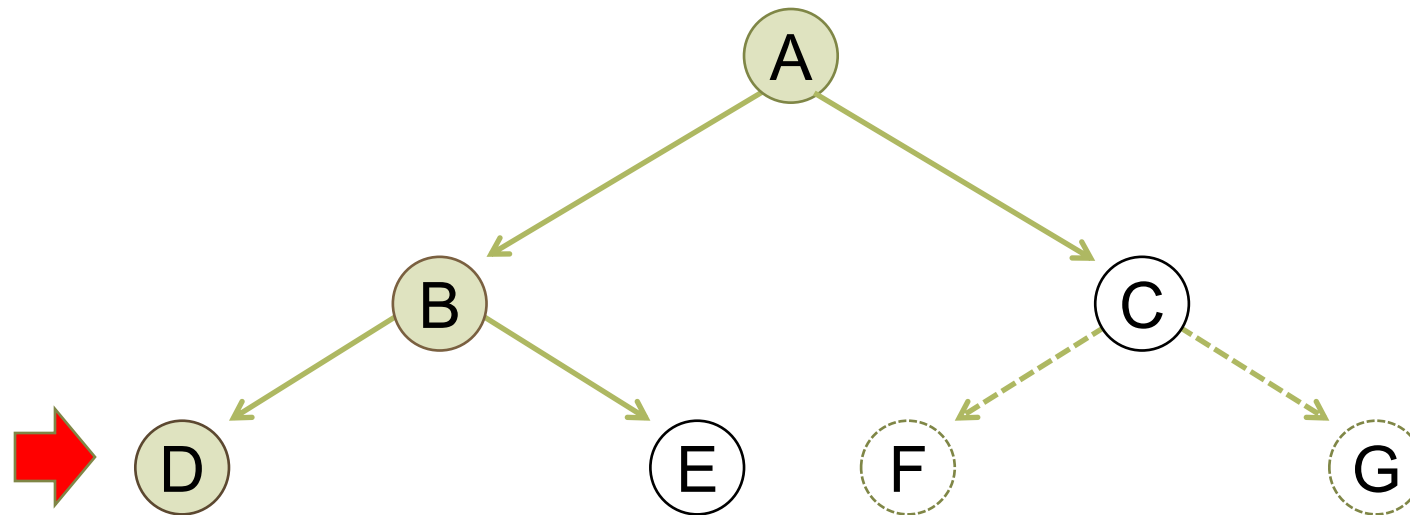
Depth-first search

- Expand deepest unexpanded node
- Implementation:
 - *fringe* = LIFO queue, i.e., put successors at front



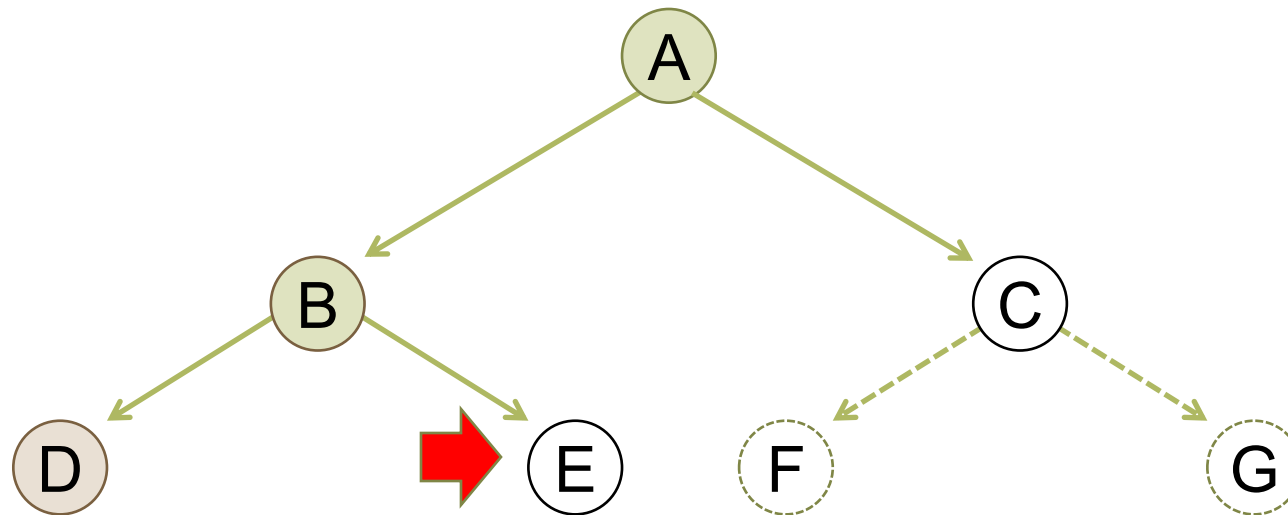
Depth-first search

- Expand deepest unexpanded node
- Implementation:
 - *fringe* = LIFO queue, i.e., put successors at front



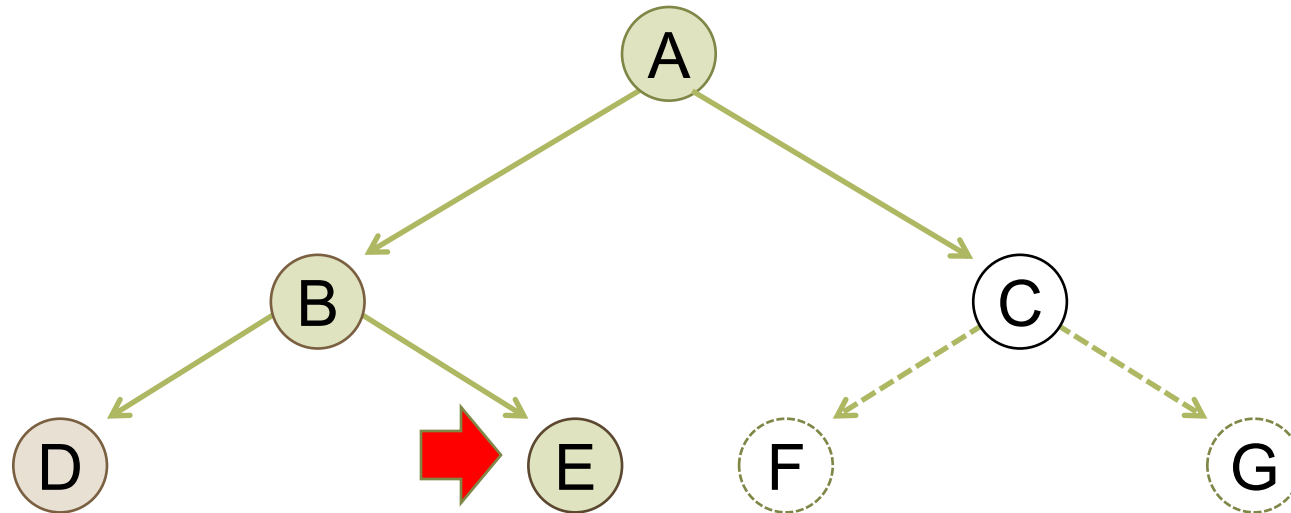
Depth-first search

- Expand deepest unexpanded node
- Implementation:
 - *fringe* = LIFO queue, i.e., put successors at front



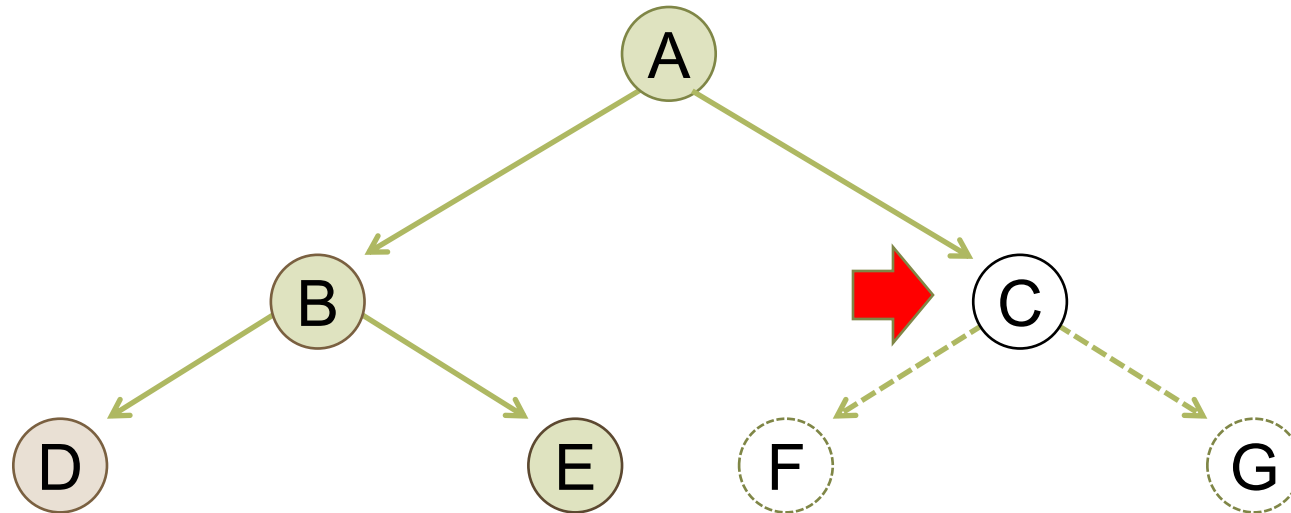
Depth-first search

- Expand deepest unexpanded node
- Implementation:
 - *fringe* = LIFO queue, i.e., put successors at front



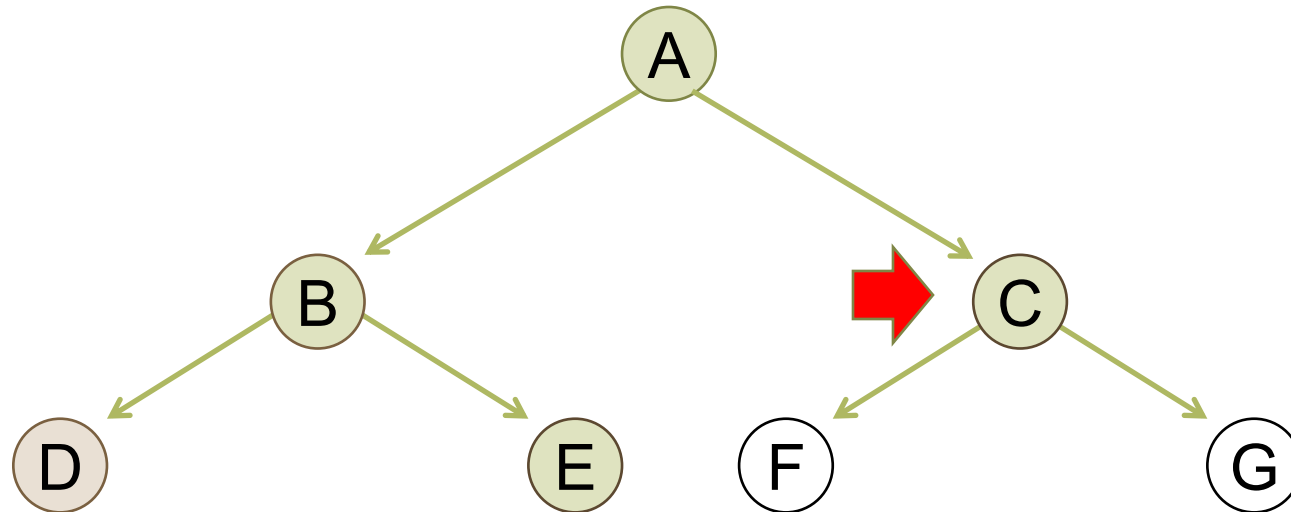
Depth-first search

- Expand deepest unexpanded node
- Implementation:
 - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

- Expand deepest unexpanded node
- Implementation:
 - *fringe* = LIFO queue, i.e., put successors at front



Properties of depth-first search

- **Complete?**

- Fails in infinite-depth spaces, spaces with loops
- Modify to avoid repeated states along path
 - complete in finite spaces

- **Optimal?**

- No – returns the first solution it finds

- **Time?**

- Could be the time to reach a solution at maximum depth of the tree, m : $O(b^m)$
- Terrible if m is much larger than d
- But if there are lots of solutions, may be much faster than BFS

- **Space?**

- $O(bm)$, i.e. linear space!

Iterative deepening search

- To keep DFS from wandering down an infinite path.
- Use DFS as a subroutine
 1. Check the root
 2. Do a DFS searching for a path of length 1
 3. If there is no path of length 1, do a DFS searching for a path of length 2
 4. If there is no path of length 2, do a DFS searching for a path of length 3...

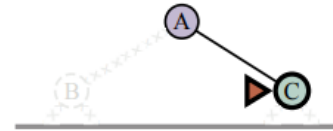
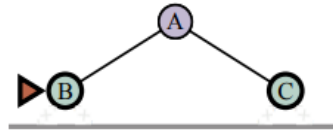
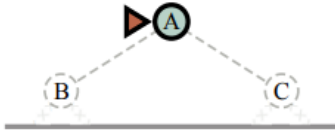
Iterative deepening search

limit: 0



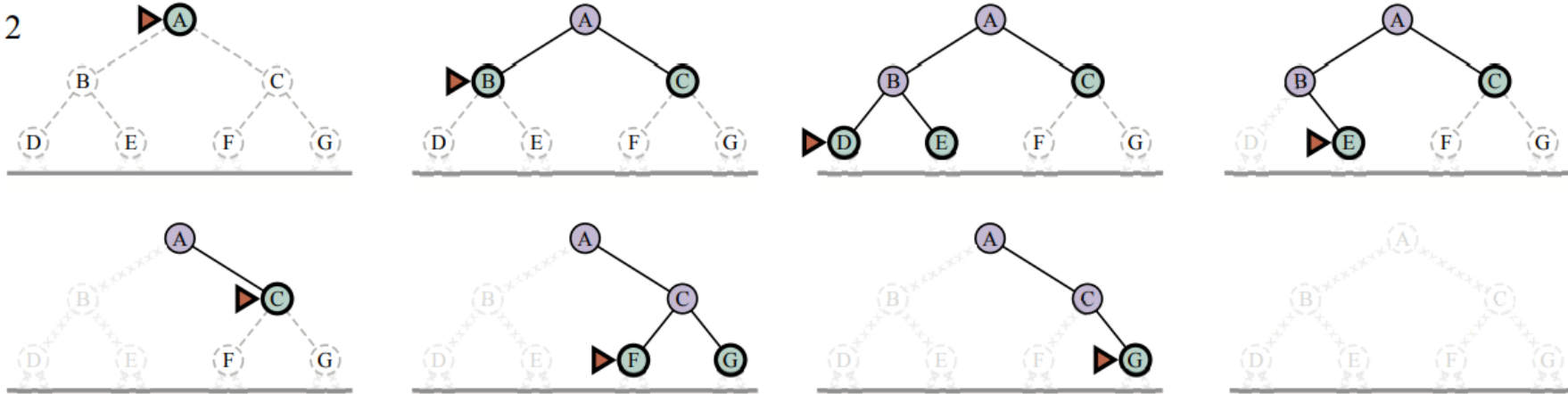
Iterative deepening search

limit: 1



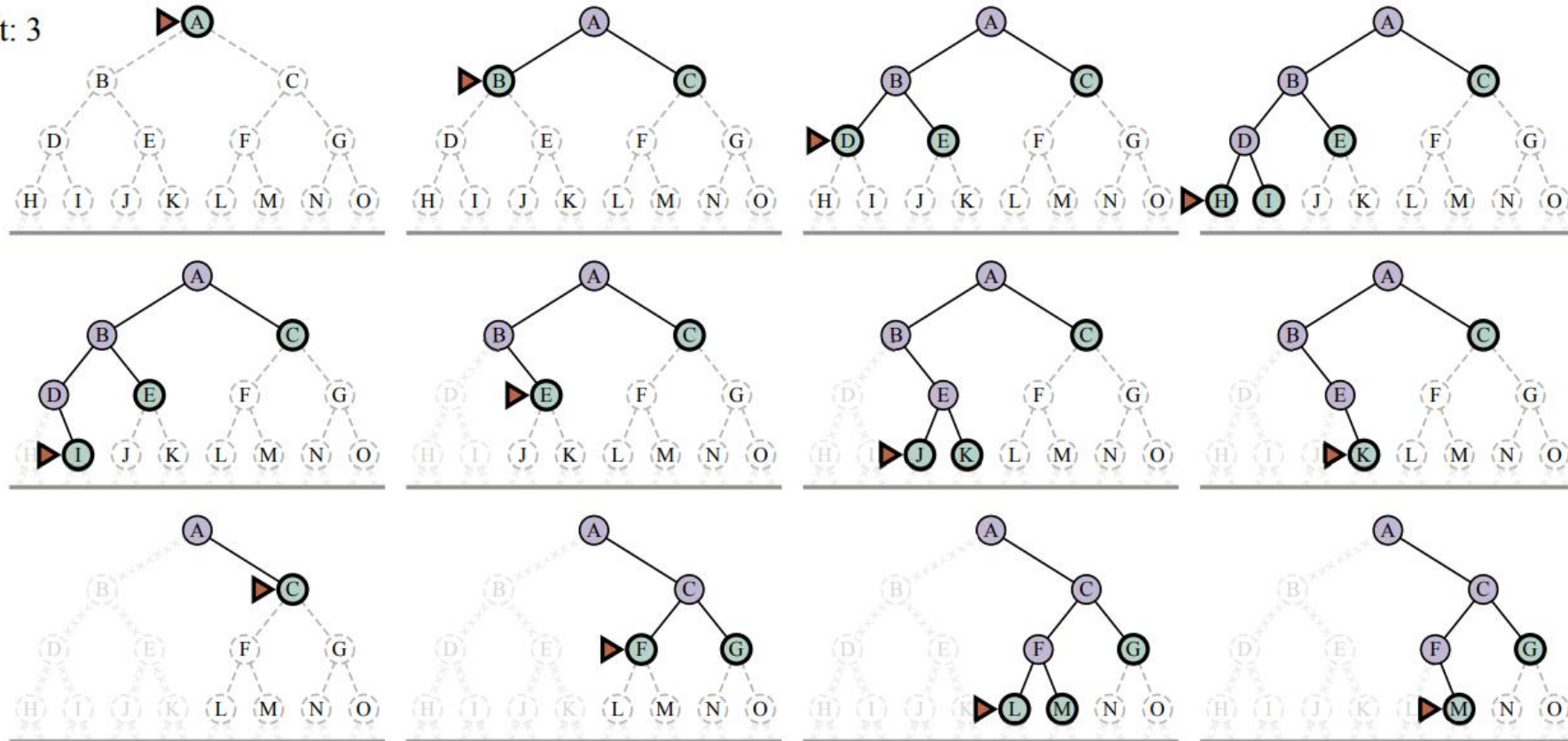
Iterative deepening search

limit: 2



Iterative deepening search

limit: 3



Properties of iterative deepening search

- **Complete?**
 - Yes
- **Optimal?**
 - Yes, if step cost = 1
- **Time?**
 - $(d+1)b^0 + d b^1 + (d-1)b^2 + \dots + b^d = O(b^d)$
- **Space?**
 - $O(bd)$

Comparing uninformed search algorithms

These two haven't covered, but you can read about them in the book

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Evaluation of search algorithms. b is the branching factor; m is the maximum depth of the search tree; d is the depth of the shallowest solution, or is m when there is no solution; ℓ is the depth limit. Superscript caveats are as follows: ¹ complete if b is finite, and the state space either has a solution or is finite. ² complete if all action costs are $\geq \epsilon > 0$; ³ cost-optimal if action costs are all identical; ⁴ if both directions are breadth-first or uniform-cost.

Next Class

- Informed Search

**Thank you for
your attention**